

AI Lab # 1

This lab introduces the foundations of Artificial Intelligence by exploring how agents perceive their environment, make decisions, and take actions to achieve goals.

Artificial Intelligence (LAB)

Instructor: Mubbashir Ahmed

Email: mubbashir.ahmed@iqra.edu.pk

Date: 26 July, 2026

About Your Instructor

Mubbashir Ahmed

Lecturer, Department of Computer Science

- MSCS (Specialization in Artificial Intelligence) from Iqra University
- Thesis in AI Agents for Business Intelligence domain
- Senior Software Engineer (Full stack) - 5+ Years of Experience
- 4+ Research papers in Artificial Intelligence and Cryptography

✉ mubbashir.ahmed@iqra.edu.pk ☎ +92 323 0221962

Class & Course Guidelines

Category	Weightage
Class Participation (Lab Manual, Assignment, Project)	35
Mid Exam	25
Final Exam	40

Bonus Points!

- Be on time
- No copied code — be ready to explain your work
- Maintain a respectful, professional environment
- Submit each assigned submissions on time

Textbooks / Reference Books

- Artificial Intelligence: A Modern Approach by Stuart Russell and Peter Norvig, 4th Edition (latest edition 2022)
- Artificial Intelligence: Foundations of Computational Agents by David L. Poole and Alan K. Mackworth, 3rd Edition (2023)
- Fuzzy Logic with Engineering Applications by Timothy J. Ross 4th Edition, John Wiley & Sons, Ltd, 2016

Today's Agenda

01

What are Agents?

02

Types of Agents

03

What are Environments?

04

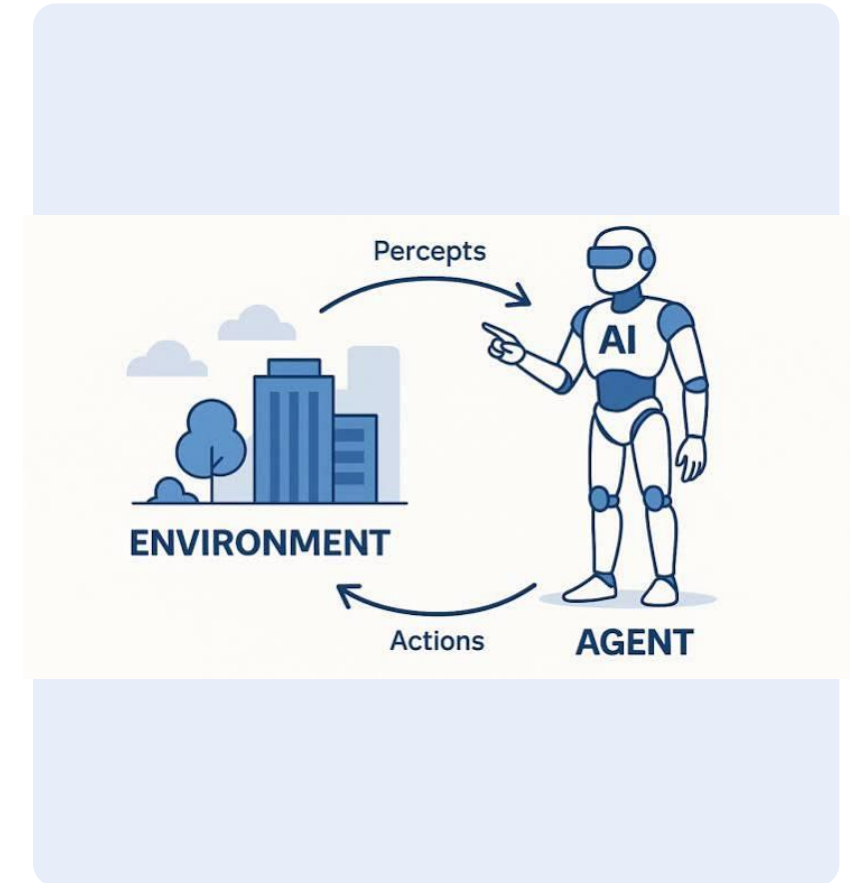
Types of Environments

05

Examples and Lab Task

What are Agents?

- An agent is something that can **observe** (see or sense) environment around it and then **do something** to reach a goal.
- Agents can be **simple or intelligent** based on their **design and capabilities**.
- **5 types:** Simple Reflex, Model-Based, Goal-Based, Utility-Based, Learning

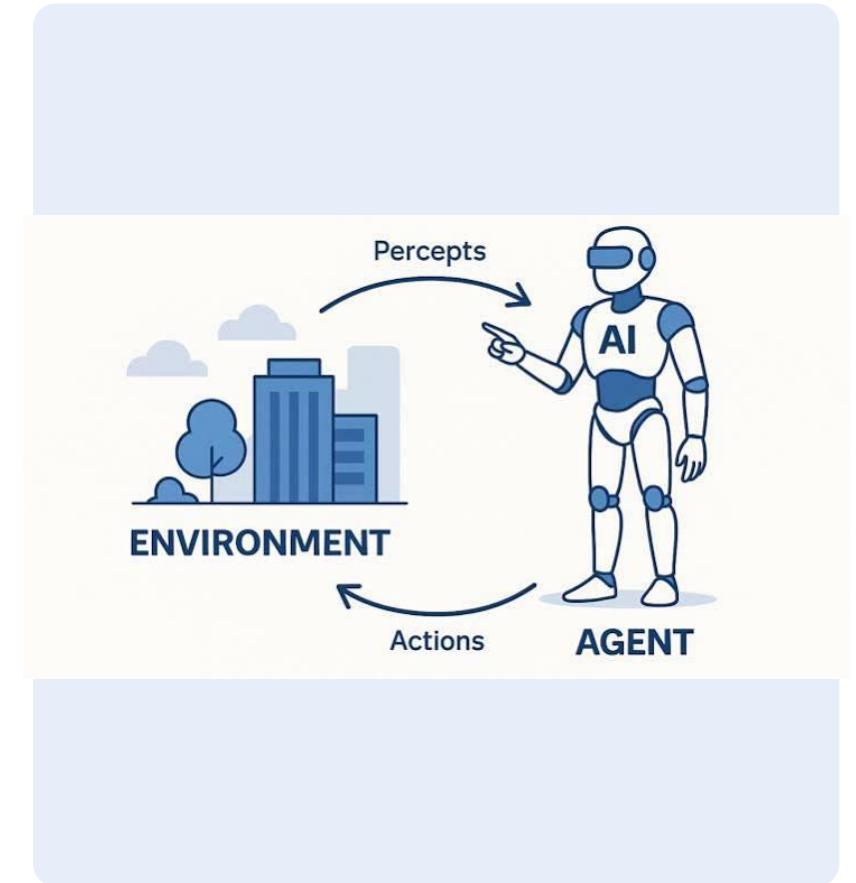


Types of Agents

Agent Type	Definition	Uses Memory?	Decision Basis	Example
Simple Reflex	Reacts to what's happening right now, without remembering anything	No	Current event only	Light turns ON when motion detected
Model-Based Reflex	Uses memory to remember past events to make better decisions	Yes	Past events + current event	Cleaning robot remembering furniture
Goal-Based	Takes actions specifically to reach a defined goal	No	Distance to goal	Robot navigating a maze
Utility-Based	Picks the best option by checking which gives more benefit	No	Best outcome/benefit	Car choosing fastest & safest route
Learning	Learns from past experiences to improve future performance	Yes	Past experience	Chatbot giving better replies over time

What are Environments?

- An **environment** is the world or situation in which an **agent** works.
- Environments can be **simple or complex** based on what the **agent** can see, do, or predict.
- **5 types:** Fully-Observable vs Partially Observable, Deterministic vs Stochastic, Static vs Dynamic, Discrete vs Continuous, Single-Agent vs Multi-Agent



Types of Environments

Environment Types		Definition	Example (A vs B)
Fully Observable	Partially Observable	Whether the agent can see the complete state of the environment or only part of it	Chess vs. self-driving car in fog
Deterministic	Stochastic	Whether the outcome of an action is certain or involves randomness	Solving a puzzle vs. rolling a dice
Static	Dynamic	Whether the environment stays the same or keeps changing while the agent decides	Crossword puzzle vs. stock market
Discrete	Continuous	Whether the environment has a limited or infinite set of actions	Tic-tac-toe vs. robotic arm movement
Single-Agent	Multi-Agent	Whether one agent acts alone or multiple agents interact/compete	Maze-solving robot vs. multiplayer online game

Examples of Agents

Now that we've covered agents and environments, let's move towards practical code-based examples

Simple Reflex Agent

Motion-based room light controller



```
def motion_based_room_light_agent(data):
    if data:
        return "Light On"
    else:
        return "Light Off"

motions = [True, False, True, False, True]
for motion in motions:
    result = motion_based_room_light_agent(motion)
    print(result)
```

Model-Based Reflex Agent

Obstacle avoidance robot



```
def obstacle_avoidance_robot_agent(data, model):
    if data in model:
        return "Avoid Obstacle"
    else:
        return "Move Forward"

obstacles = ["chair", "table", "wall", "door", "window"]
model = ["chair", "table", "wall"]
for obstacle in obstacles:
    result = obstacle_avoidance_robot_agent(obstacle, model)
    print(result)
```

Goal-Based Agent

Maze solver robot



```
def maze_solver_robot_agent(data, goal):
    if data == goal:
        return "Reached Goal"
    else:
        return "Move Forward"

maze = ["A", "B", "C", "D", "E"]
goal = "E"
for maze in maze:
    result = maze_solver_robot_agent(maze, goal)
    print(result)
```

Utility-Based Agent

Car fastest route finder



```
def car_fastest_route_finder_agent(paths):  
    best_path = min(paths, key = paths.get)  
    return best_path  
  
paths = {"A": 10, "B": 20, "C": 30, "D": 40, "E": 50}  
result = car_fastest_route_finder_agent(paths)  
print(result)
```

Learning Agent

Rule-based chatbot



```
known_responses = {"Hello": "Hello, how can I help you?", "Goodbye": "Goodbye, have a great day!", "Thank you": "You're welcome!"}

def rule_based_chatbot_agent(data):
    if data in known_responses:
        return known_responses[data]
    else:
        new_reply = input(f"I dont know the answer to '{data}'. Please teach me: ")
        known_responses[data] = new_reply
        return new_reply

data = ["Hello", "What is the capital of Pakistan?", "Thank you", "What is the capital of Pakistan?", "Goodbye", "What is the capital of India?"]

for data in data:
    result = rule_based_chatbot_agent(data)
    print(f"{data}: {result}")
```

Lab Task: Elevator Reflex Agent (Single-Agent in a Fully Observable, Deterministic, Static Environment)

An elevator agent operates in a building with multiple floors. The agent perceives its current floor and the requested floor, and must decide its next action accordingly. The agent can perform the following actions:

- *Move Up – move the elevator to a higher floor*
- *Move Down – move the elevator to a lower floor*
- *Open Doors – open the doors when the elevator reaches the requested floor*

Write a Python program that implements a Simple Reflex Agent for the elevator problem. The agent should:

1. *Take the current floor and requested floor as input.*
2. *Apply a simple if-else rule to decide the next action:*
 - *If requested floor > current floor → Move Up*
 - *If requested floor < current floor → Move Down*
 - *If requested floor = current floor → Open Doors*
3. *Print the action taken for each request.*
4. *Test your agent with at least three different (current floor, requested floor) pairs.*

Thank You

Questions or discussions are welcomed!